

Контролери

До момента имаме 3 модела в папка Models:

Кодът на Employee.cs:

```
namespace EmployeeManagement.Models
{
    public class Employee
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public string Email { get; set; }

        public string Department { get; set; }
    }
}
```

Кодът на IEmployeeRepository.cs:

```
namespace EmployeeManagement.Models
{
    public interface IEmployeeRepository
    {
        public Employee GetEmployee(int id);
    }
}
```

Кодът на MockEmployeeRepository.cs:

```
using System.Collections.Generic;
using System.Linq;

namespace EmployeeManagement.Models
{
    public class MockEmployeeRepository : IEmployeeRepository
    {
        private List<Employee> _employeeList;
        public MockEmployeeRepository()
        {
            _employeeList = new List<Employee>()
            {
                new Employee() { Id = 1, Name = "Мария", Department = "Biology",
Email = "Maria@omg.bg" },
            }
        }
    }
}
```

```

        new Employee() { Id = 2, Name = "Иван", Department = "Physics",
Email = "Ivan@omg.bg" },
        new Employee() { Id = 3, Name = "Стефан", Department = "Math",
Email = "Stefan@omg.bg" }
    };
}
public Employee GetEmployee(int id)
{
    // return _employeeList[id];
    return _employeeList.FirstOrDefault( e => e.Id == id);
}
}
}

```

Към момента, цялата работа се извършва от клас MockEmployeeRepository, който съхранява логиката.

За да работят двата модела, трябва да укажем, че ги свързваме в Startup.cs.

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore(option => option.EnableEndpointRouting = false);
    services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
}

```

Тази връзка/зависимост се нарича Dependency Injection Container и се добавя необходимата библиотека за нея чрез Ctrl+.

ConfigureService отговаря за вградените services във framework + нашите custom. В него параметър services е от тип IServiceCollection, който е интерфейс. Благодарение на него можем да добавим връзка тип Singleton между първо интерфейса IEmployeeRepository – и второ, имплементацията на този интерфейс MockEmployeeRepository.

Сверяваме необходимите библиотеки – в сиво са новите:

Startup.cs 1 X

HomeController.cs

EmployeeManagement > Startup.cs > Startup > ConfigureServices

```
1  using EmployeeManagement.Models;
2  using Microsoft.AspNetCore.Builder;
3  using Microsoft.AspNetCore.Hosting;
4  using Microsoft.AspNetCore.Http;
5  using Microsoft.AspNetCore.Mvc;
6  using Microsoft.Extensions.Configuration;
7  using Microsoft.Extensions.DependencyInjection;
8  using Microsoft.Extensions.Hosting;
9  using Microsoft.Extensions.Logging;
10 using System;
11 using System.Collections.Generic;
12 using System.Linq;
13 using System.Threading.Tasks;
```

За да сработи целият код, трябва моделите да бъдат извикани, за да изпълнят предназначените си действия в контролера.

Към момента, в HomeController имаме 2 метода, които преименуваме, за да не ни пречат на MVC и Data. Можем да ги достъпим съответно от localhost:5000/Home/MVC и localhost:5000/Home/Data

up.cs 1 HomeController.cs •

eeManagement > Controllers > HomeController.cs > ...

```
using EmployeeManagement.Models; библиотеки
using Microsoft.AspNetCore.Mvc;

namespace EmployeeManagement.Controllers
{
    0 references
    public class HomeController: Controller
    {
        0 references
        public string MVC()
        {
            return "Hello from MVC!";
        }

        0 references
        public JsonResult Data()
        {
            return Json (new {id=1, name="XII class!"});
        }
    }
}
```

Над тях можем да напишем още един action по подразбиране index.

```
private IEmployeeRepository _employeeRepository; field
public HomeController(IEmployeeRepository employeeRepository) ctor
{
    _employeeRepository = employeeRepository;
}

0 references
public string Index()
{
    return _employeeRepository.GetEmployee(1).Name;
} method
```

Целият HomeController.cs:

```
using EmployeeManagement.Models;
```

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace EmployeeManagement.Controllers
{
    public class HomeController: Controller
    {
        private IEmployeeRepository _employeeRepository;

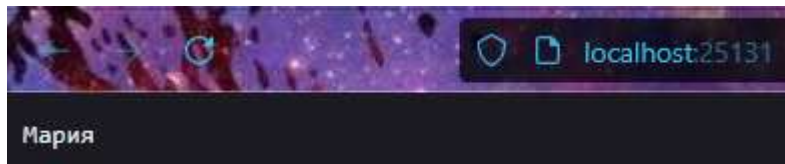
        public HomeController(IEmployeeRepository employeeRepository)
        {
            _employeeRepository = employeeRepository;
        }

        public string Index()
        {
            return _employeeRepository.GetEmployee(1).Name;
        }

        public string MVC()
        {
            return "Hello from MVC!";
        }

        public JsonResult Data()
        {
            return Json (new {id=1, name="XII class!"});
        }
    }
}
```

Резултат:



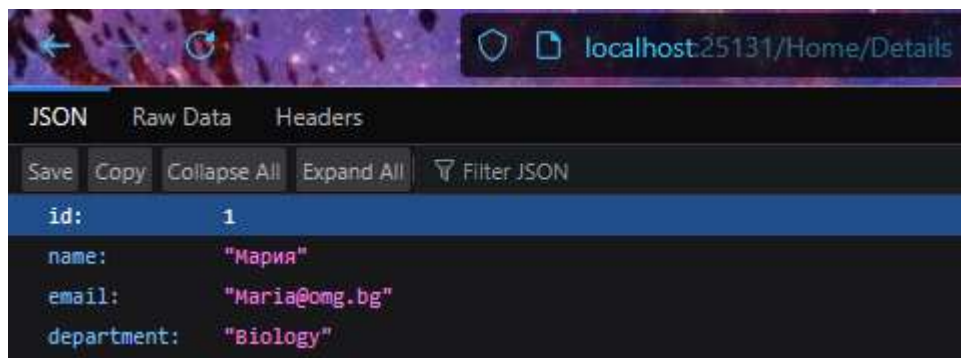
За да предотвратим инцидентно добавяне на променлива, се застраховаме като добавяме readonly

```
private readonly IEmployeeRepository _employeeRepository;
```

Може да добавим още един Action, който да изведе всички hard-code данни като познатия json:

```
public JsonResult Details()
{
    Employee model = _employeeRepository.GetEmployee(1);
    return Json(model);
}
```

Съответно, ще го намерим на адрес /Home/Details:



Нека изведем тези данни като отделна страница View. За тази цел, променяме типа данни да не бъдат Json -> View и сменяме името на Action с друго, за да не се повтаря.

```
0 references
public JsonResult Details() copy
{
    Employee model = _employeeRepository.GetEmployee(1);
    return Json(model);
}

0 references
public ViewResult DetailsPage() paste
{
    Employee model = _employeeRepository.GetEmployee(1);
    return View(model);
}
```



An unhandled exception occurred while processing the request.

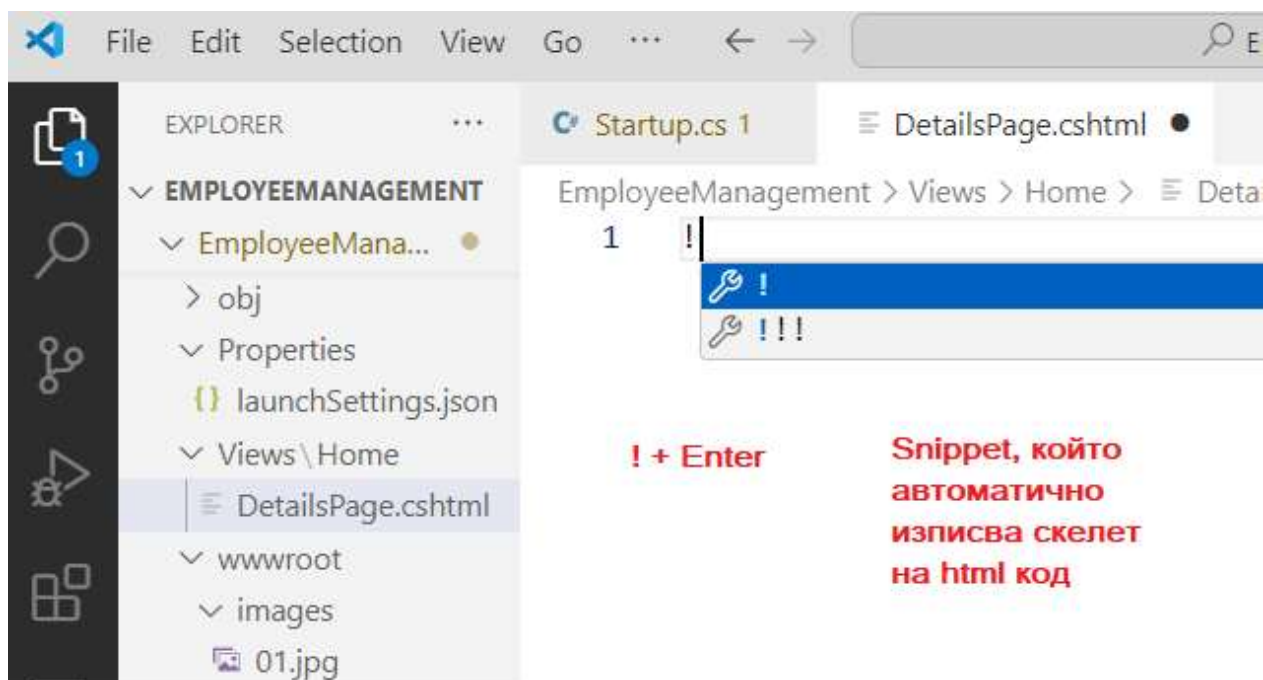
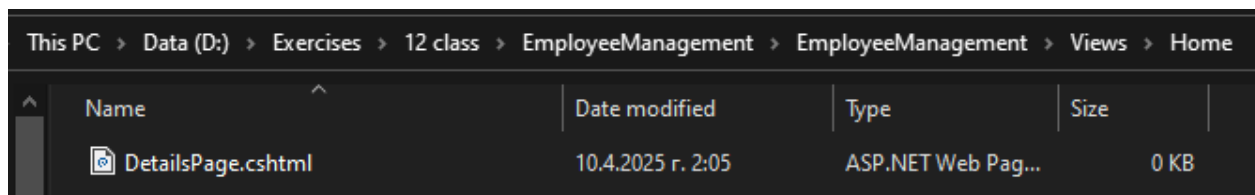
InvalidOperationException: No service for type 'Microsoft.AspNetCore.Mvc.ViewFeatures.ITempDataDictionaryFactory' has been registered.
Microsoft.Extensions.DependencyInjection.ServiceProviderServiceExtensions.GetRequiredService(IServiceProvider provider, Type serviceType)

```
tory.cs Startup.cs HomeController.cs
EmployeeManagement.Startup ConfigureServices(IServiceCollection services)
0 references
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore(option => option.EnableEndpointRouting = false);
    services.AddSingleton<IEmployeeRepository, MockEmployeeRepository>();
}
```

Сега вече извежда съвсем друга грешка, че търси страницата в съответните директории една след друга и не я открива.



Съответно трябва да я създадем и поставим в правилните папки:

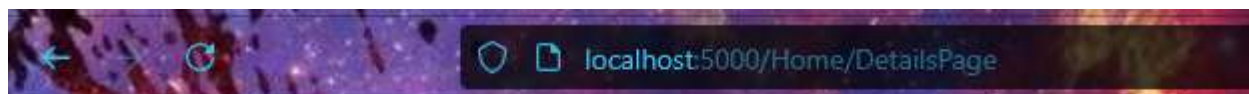


Готово за писане в body:


```
Startup.cs 1  DetailsPage.cshtml
EmployeeManagement > Views > Home > DetailsPage.cshtml
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9      |
10 </body>
11 </html>
```

Поставяме заглавие: `<h1>Hello from Views/Home/DetailsPage.cshtml!</h1>`

Вече като дадем Refresh, я намира:

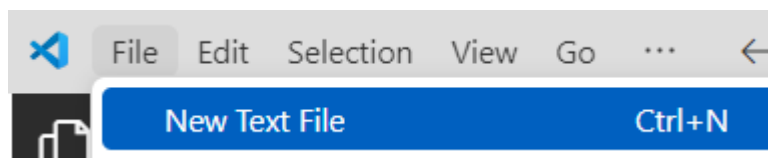


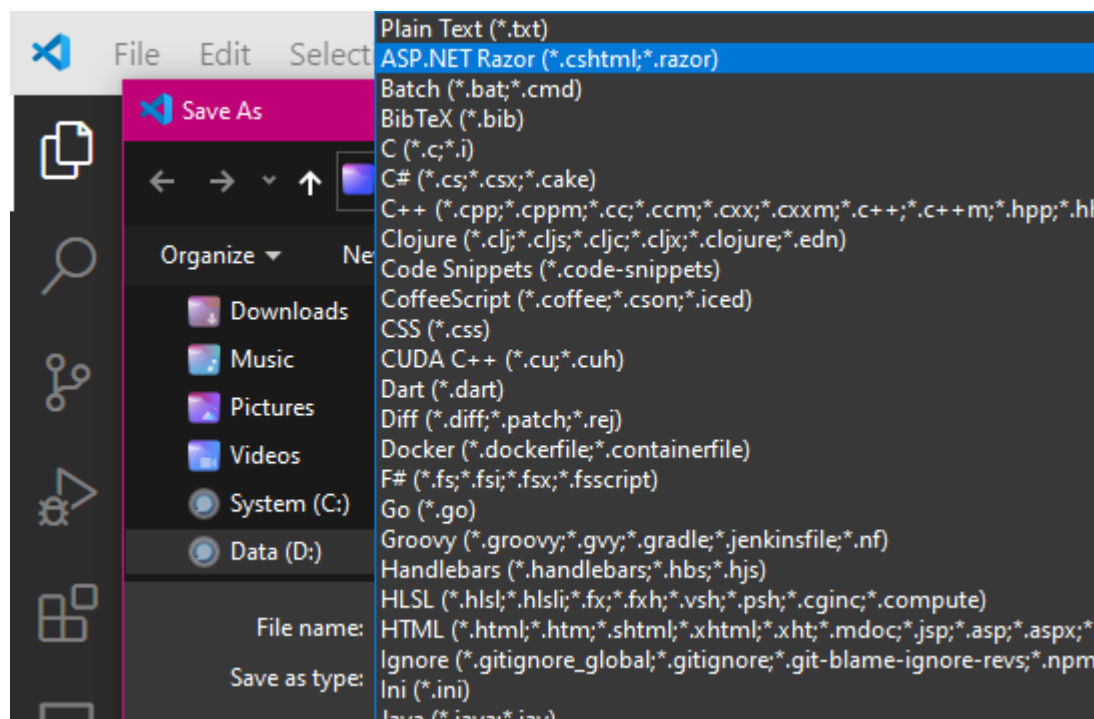
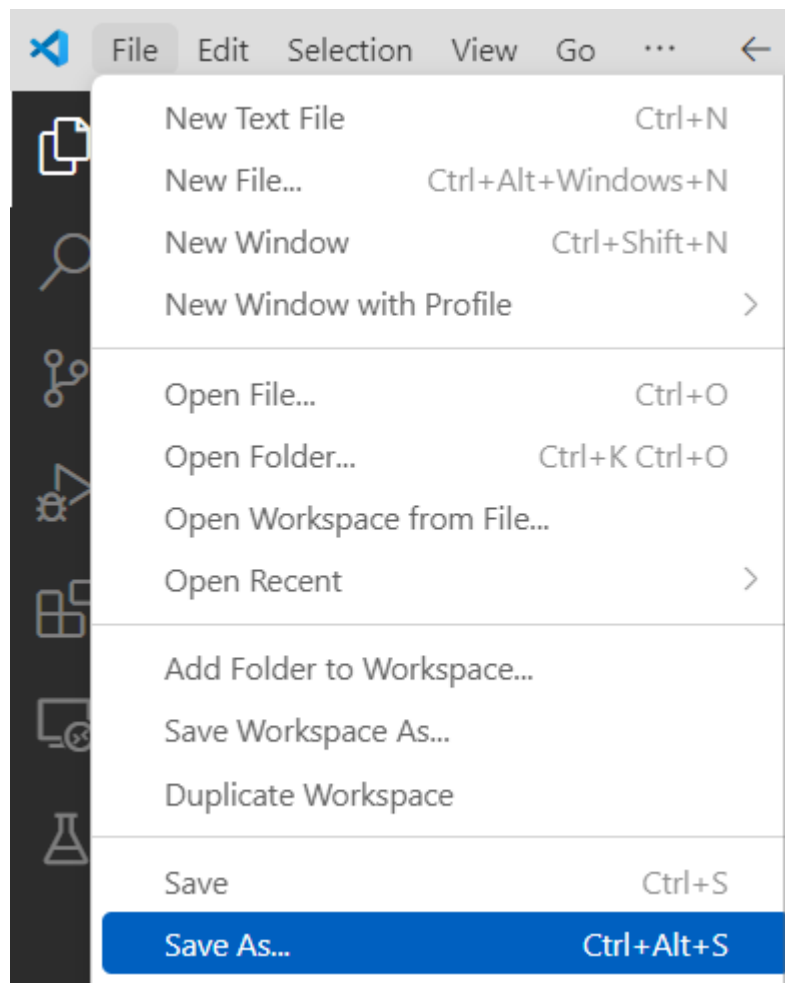
Hello from Views/Home/DetailsPage.cshtml!

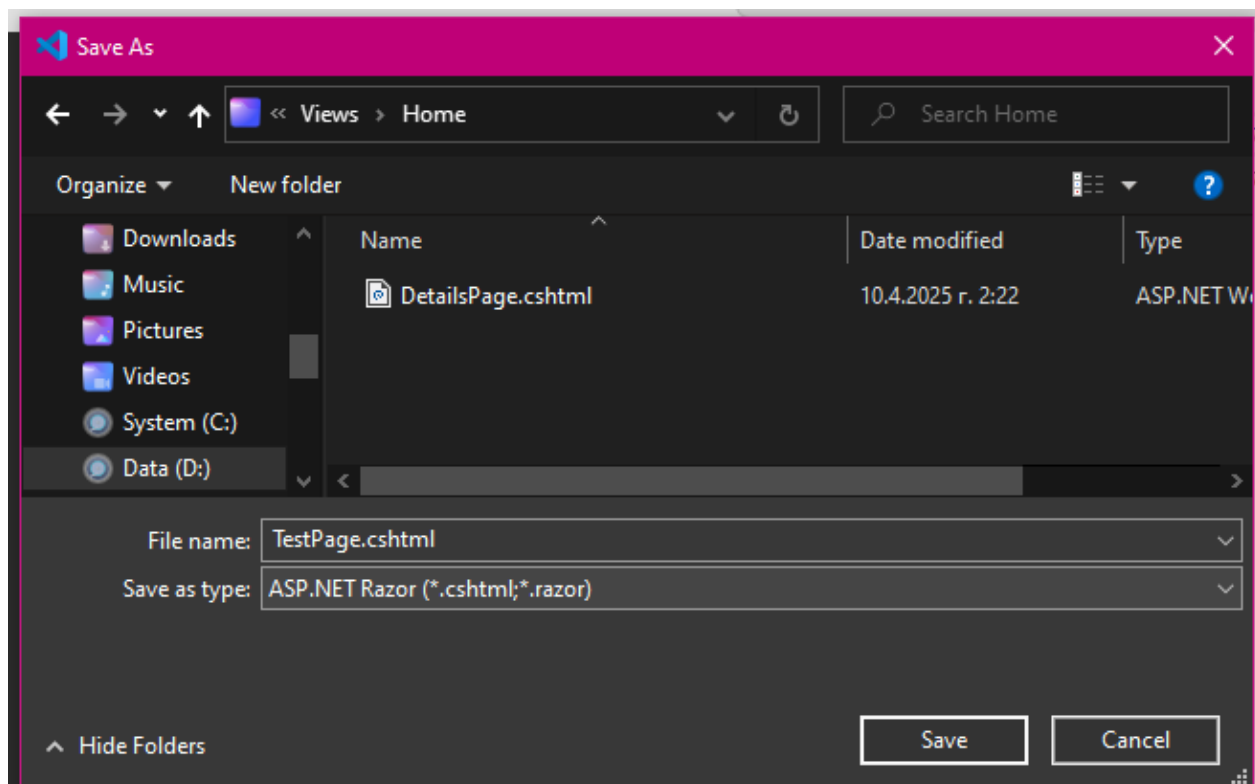
Нека направим тестов Action в HomeController:

```
0 references
public ActionResult TestPage()
{
    return View();
}
```

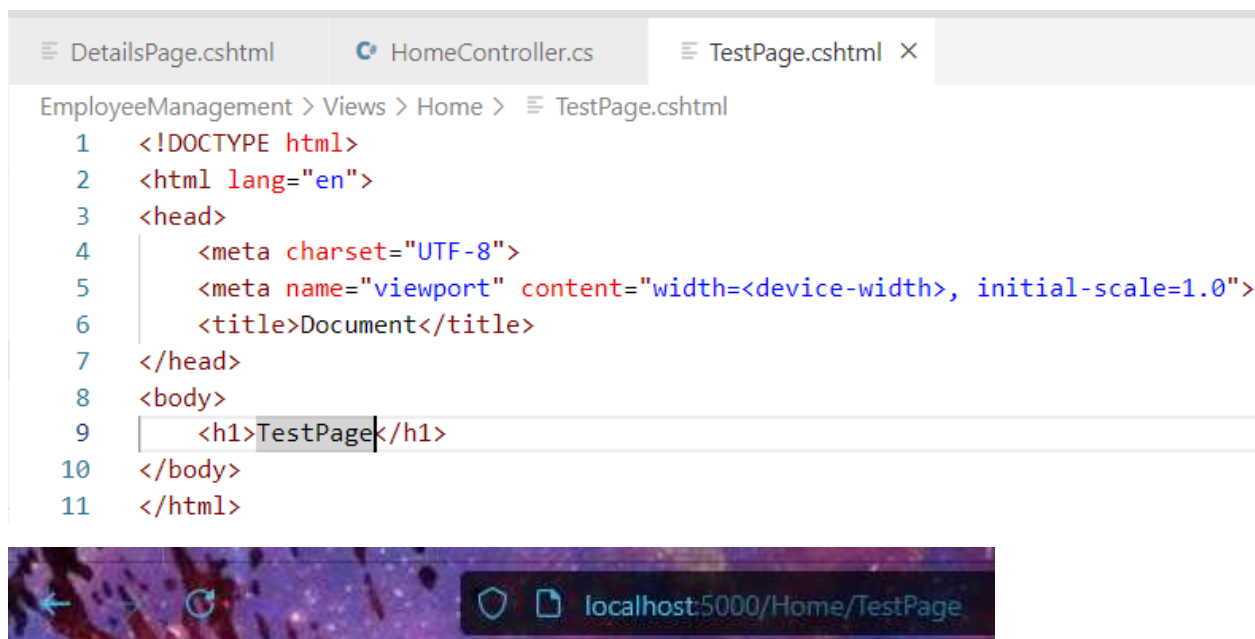
Значи ще търси страница във Views/Home/TestPage.cshtml, която трябва да създадем:







Отново може да използваме бърз снипет и сложим някакво заглавие:



TestPage

Видяхме, че ако е просто `return View ();` без да пише нищо, търси файл със същото име като Action в същата папка като името на контролера.

Ако напишем в скобите запишем в кавички друго име, ще търси него:

```
public ActionResult TestDifferent()  
{  
    return View("Different");  
}
```

Записваме един файл с име TestDifferent.cshtml



Hello from different name file!

Търси адреса на action и там изобразява файл с друго име.

Ако направим за проверка другото – отидем в <http://localhost:5000/Home/Different> няма да намери страницата.

Последен тест: можем да се качим 1 директория нагоре с ../

```
public ActionResult TestUp()  
{  
    return View("../Up.cshtml");  
}
```

Което означава, че няма да търси файла във Views/Home/Up.cshtml, а една папка нагоре във Views/Up.cshtml

По същият начин могат да се търсят и използват:

./ - една директория навътре

../ - една директория нагоре

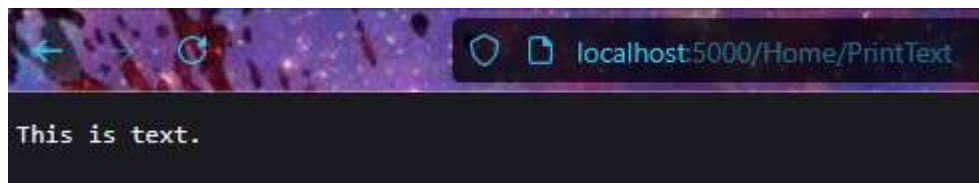
../../ - три директории нагоре

~/ - главната root директория, независимо колко нива нагоре е.

Като цяло, всички те важат и в статичния .html, но знакът ~ дава грешки в .html, а в .cshtml дава грешки.

Можем да връщаме и текст, и число, може да подаваме променливи от други места. Важното е типът, който декларираме, че връща метода при името си, да съвпада с върнатия резултат на return.

```
public string PrintText()  
{  
    return "This is text.";  
}
```



Макар, че не е особено правилно, може да подаваме и число:

```
public int PrintValue()  
{  
    return 3;  
}
```

